Table 1: Agents in the Proposed Agentic Graph Construction Framework

ID	**Agent Name**	**Responsibility**
A1	Document Ingestion Agent	Validates the structured JSON document, extracts metadata, registers the document, and initializes the graph construction workflow.
A2	Semantic Chunking Agent	Partitions the document into semantically coherent overlapping chunks while preserving section hierarchy and contextual continuity.
A3	Triple Extraction Agent	Extracts knowledge triples (Head, Relation, Tail, Confidence) from each semantic chunk using an LLM.
A4	Coreference Resolution Agent	Resolves aliases, abbreviations, pronouns, and duplicate mentions.
A5	Confidence Evaluation Agent	Filters unreliable triples according to a confidence threshold.
A6	Entity Resolution Agent	Matches extracted entities with existing graph nodes.
A7	Relation Normalization Agent	Maps extracted relations into the predefined ontology.
A8	Conflict Detection Agent	Identifies New, Merge, Update, Conflict, and Reject operations.
A9	Decision Agent	Determines the graph operation according to validation rules.
A10	Graph CRUD Agent	Creates, updates, merges, and deletes graph nodes and relationships.
A11	Provenance Agent	Stores provenance metadata for every node and relationship.
A12	Embedding Agent	Generates semantic embeddings for entities and document chunks.
A13	Vector Index Agent	Stores embeddings in the vector database.
A14	Construction Validation Agent	Validates graph consistency and ontology compliance.
A15	Graph Health Audit Agent	Detects duplicate entities, orphan nodes, stale edges, and graph anomalies.

Algorithm 1: Overall Agentic Graph Construction Framework

Input: Structured JSON J, Metadata M, Existing Knowledge Graph KG, Vector Database VDB

Output: Updated Knowledge Graph KG', Updated Vector Database VDB', Construction Report R

Receive structured JSON document J and metadata M;
Invoke **Document Ingestion Agent**[A1];
Invoke **Semantic Chunking Agent**[A2];
foreach *Semantic Chunk* C_i **do**
    Invoke **Triple Extraction Agent**[A3];
    Invoke **Coreference Resolution Agent**[A4];
    Invoke **Confidence Evaluation Agent**[A5];
    if *Confidence Score* $< \tau$ **then**
        Discard extracted triples;
        Continue to the next chunk;
    else
        Invoke **Entity Resolution Agent**[A6];
        Invoke **Relation Normalization Agent**[A7];
        Invoke **Conflict Detection Agent**[A8];
        Invoke **Decision Agent**[A9];
        if *Decision* = *New* **then**
            Invoke **Graph CRUD Agent**[A10] to create graph nodes and relationships;
        else if *Decision* = *Merge* **then**
            Merge entities and relationships into the existing graph;
        else if *Decision* = *Update* **then**
            Update confidence scores and provenance metadata;
        else
            Reject conflicting knowledge;
        Invoke **Provenance Agent**[A11];
        Invoke **Embedding Agent**[A12];
        Invoke **Vector Index Agent**[A13];
Invoke **Construction Validation Agent**[A14];
Invoke **Graph Health Audit Agent**[A15];
Export updated Knowledge Graph KG';
Export updated Vector Database VDB';
Generate Construction Report R;
return (KG', VDB', R)

Algorithm 2: Document Ingestion Agent

Input: Structured JSON document J, Metadata M
Output: Validated Document Object D

Receive structured JSON document J;
Validate the JSON schema;
if *Schema is valid* **then**
    Extract document metadata;
    Generate a unique Document ID;
    Register the document in the graph repository;
    Initialize the graph construction session;
    Create validated document object D;
    Forward D to the **Semantic Chunking Agent**[A2];
else
    Generate validation error report;
    Reject the document;
    Terminate graph construction;
return D

Algorithm 3: Semantic Chunking Agent

Input: Validated Document Object D
Output: Semantic Chunks $\{C_1, C_2, \ldots, C_n\}$

Receive validated document object D;
Identify document sections and headings;
Split document into semantic chunks;
Maintain overlapping context between adjacent chunks;
Assign unique Chunk ID to each chunk;
Preserve section hierarchy and metadata;
Forward semantic chunks to the **Triple Extraction Agent**[A3];
return $\{C_1, C_2, \ldots, C_n\}$

Algorithm 4: Triple Extraction Agent

Input: Semantic Chunk C_i
Output: Knowledge Triples T

Receive semantic chunk C_i;
Prompt the LLM with extraction instructions;
Extract knowledge triples as
($Head$, $Relation$, $Tail$, $Confidence$);
Normalize extracted triples;
Assign confidence scores;
Forward triples to the **Coreference Resolution Agent**[A4];
return T

Algorithm 5: Coreference Resolution Agent

Input: Knowledge Triples T
Output: Resolved Triples T'

Receive extracted triples;
Identify pronouns and aliases;
Resolve abbreviations to canonical names;
Merge duplicate entity mentions;
Replace references with canonical entities;
Forward resolved triples to the **Confidence Evaluation Agent**[A5];
return T'

Algorithm 6: Confidence Evaluation Agent

Input: Resolved Triples T'
Output: Validated Triples T_v

Receive resolved triples;
foreach *Triple* t_i **do**
    Evaluate confidence score;
    if *Confidence* $\geq \tau$ **then**
        Accept triple;
        Store into validated set;
    else
        Discard triple;
Forward validated triples to the **Entity Resolution Agent**[A6];
return T_v

Algorithm 7: Entity Resolution Agent

Input: Validated Triples T_v, Existing Knowledge Graph KG
Output: Canonical Entities

Receive validated triples;
foreach *Entity* e_i **do**
    Search matching entity in Knowledge Graph;
    Compute semantic similarity;
    if *Similarity* $\geq \delta$ **then**
        Link entity to existing graph node;
    else
        Create a new graph entity;
Forward canonical entities to the **Relation Normalization Agent**[A7];
return *Canonical Entities*

Algorithm 8: Relation Normalization Agent

Input: Canonical Entities, Validated Triples T_v, Ontology O
Output: Normalized Triples T_n

Receive validated triples;
foreach *Relation* r_i **do**
    Search ontology for equivalent relation;
    Normalize relation to ontology-defined predicate;
    Replace synonymous relations with canonical relation;
Forward normalized triples to the **Conflict Detection Agent**[A8];
return T_n

Algorithm 9: Conflict Detection Agent

Input: Normalized Triples T_n, Knowledge Graph KG
Output: Conflict Status

Receive normalized triples;
foreach *Triple* t_i **do**
    Search corresponding relationship in KG;
    if *Triple does not exist* **then**
        Mark as New;
    else
        Compare entity, relation, confidence, and provenance;
        Determine whether Merge, Update, Conflict, or Reject;
Forward conflict status to the **Decision Agent**[A9];
return *Conflict Status*

Algorithm 10: Decision Agent

Input: Conflict Status
Output: Graph Operation

Receive conflict analysis;
foreach *Triple* t_i **do**
    if *Status* = *New* **then**
        Decision ← Create;
    else
        if *Status* = *Merge* **then**
            Decision ← Merge;
        else if *Status* = *Update* **then**
            Decision ← Update;
        else
            Decision ← Reject;
Forward decision to the **Graph CRUD Agent**[A10];
return *Decision*

Algorithm 11: Graph CRUD Agent

Input: Decision, Normalized Triples
Output: Updated Knowledge Graph

Receive graph operation decision;
foreach *Decision* d_i **do**
    if d_i = *Create* **then**
        Insert new node and relationship;
    else if d_i = *Merge* **then**
        Merge duplicate nodes and relationships;
    else if d_i = *Update* **then**
        Update existing relationship confidence;
    else
        Discard conflicting knowledge;
Forward updated graph to the **Provenance Agent**[A11];
return *Updated Knowledge Graph*

Algorithm 12: Provenance Agent

Input: Updated Knowledge Graph
Output: Knowledge Graph with Provenance

Receive updated graph;
Attach document identifier;
Attach chunk identifier;
Attach page number;
Attach timestamp;
Attach confidence score;
Attach source reference;
Store provenance metadata for every node and relationship;
Forward graph to the **Embedding Agent**[A12];
return *Knowledge Graph with Provenance*

Algorithm 13: Embedding Agent

Input: Knowledge Graph with Provenance
Output: Entity and Chunk Embeddings

Receive knowledge graph with provenance;
foreach *Entity and Document Chunk* **do**
    Generate dense semantic embedding;
    Associate embedding with corresponding graph node;
Forward embeddings to the **Vector Index Agent**[A13];
return *Entity and Chunk Embeddings*

Algorithm 14: Vector Index Agent

Input: Entity and Chunk Embeddings, Vector Database VDB
Output: Updated Vector Database

Receive generated embeddings;
foreach *Embedding* e_i **do**
    Insert or update embedding in the vector database;
    Store mapping between embedding and graph entity;
    Verify successful indexing;
Forward updated graph to the **Construction Validation Agent**[A14];
return *Updated Vector Database*

Algorithm 15: Construction Validation Agent

Input: Knowledge Graph KG, Vector Database VDB
Output: Validated Knowledge Graph

Receive updated knowledge graph and vector database;
Validate ontology compliance;
Check node and relationship consistency;
Verify graph connectivity;
Detect invalid entities and relations;
Validate vector index mappings;
Forward validated graph to the **Graph Health Audit Agent**[A15];
return *Validated Knowledge Graph*

Algorithm 16: Graph Health Audit Agent

Input: Validated Knowledge Graph
Output: Construction Report R

Receive validated knowledge graph;
Detect orphan nodes;
Detect duplicate entities;
Detect stale relationships;
Detect disconnected graph components;
Identify low-confidence edges;
Generate graph quality metrics;
Generate construction report R;
Export updated knowledge graph and audit report;
return R